

1. Java Program to Swap Two Numbers

// This program swaps two numbers using a temporary variable

```
public class SwapNumbers {  
    public static void main(String[] args) {  
        int a = 5, b = 10;  
        System.out.println("Before swapping: a = " + a + ", b = " + b);  
  
        int temp = a; // Store value of 'a' in temp  
        a = b; // Assign value of 'b' to 'a'  
        b = temp; // Assign value of temp (original 'a') to 'b'  
  
        System.out.println("After swapping: a = " + a + ", b = " + b);  
    }  
}
```

2. Convert Fahrenheit to Celsius

// This program converts temperature from Fahrenheit to Celsius

```
import java.util.Scanner;  
  
public class TemperatureConverter {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Enter temperature in Fahrenheit: ");  
        double fahrenheit = scanner.nextDouble();  
  
        double celsius = (fahrenheit - 32) * 5/9; // Conversion formula  
        System.out.println("Temperature in Celsius: " + celsius);  
  
        scanner.close();  
    }  
}
```

3. Swap Two Numbers Without Using a Third Variable

// This program swaps two numbers without using a third variable

```
public class SwapWithoutTemp {  
    public static void main(String[] args) {  
        int x = 7, y = 14;
```

```
        System.out.println("Before swapping: x = " + x + ", y = " + y);

        x = x + y; // Add both numbers
        y = x - y; // Subtract new 'x' from old 'y'
        x = x - y; // Subtract new 'y' from new 'x'

        System.out.println("After swapping: x = " + x + ", y = " + y);
    }
}
```

4. Java Program to Add Two Numbers

```
// This program adds two numbers
import java.util.Scanner;

public class AddNumbers {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int num1 = scanner.nextInt();

        System.out.print("Enter second number: ");
        int num2 = scanner.nextInt();

        int sum = num1 + num2; // Adding numbers
        System.out.println("Sum: " + sum);

        scanner.close();
    }
}
```

5. Object-Oriented Programming Example

```
// This program demonstrates object-oriented programming by defining a class and creating
objects
class Car {
    String brand;
    int speed;

    // Constructor to initialize car details
```

```

    Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    void display() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
    }
}

public class OOPEXample {
    public static void main(String[] args) {
        Car car1 = new Car("Toyota", 180);
        Car car2 = new Car("Honda", 200);

        car1.display();
        car2.display();
    }
}

```

6. Constructors

```

// This program demonstrates constructors in Java
class Student {
    String name;
    int age;

    // Constructor to initialize student details
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void showDetails() {
        System.out.println("Student Name: " + name + ", Age: " + age);
    }
}

public class ConstructorExample {
    public static void main(String[] args) {
        Student student1 = new Student("Venkatesh", 21);
        student1.showDetails();
    }
}

```

```
}
```

7. Abstract Classes

// This program demonstrates abstract classes in Java

```
abstract class Animal {  
    abstract void makeSound(); // Abstract method
```

```
    void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

```
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark Bark!");  
    }  
}
```

```
public class AbstractClassExample {  
    public static void main(String[] args) {  
        Dog myDog = new Dog();  
        myDog.makeSound();  
        myDog.sleep();  
    }  
}
```

8. Interfaces

// This program demonstrates interfaces in Java

```
interface Vehicle {  
    void start(); // Abstract method  
}
```

```
class Car implements Vehicle {  
    public void start() {  
        System.out.println("Car is starting...");  
    }  
}
```

```
public class InterfaceExample {
```

```
public static void main(String[] args) {  
    Car myCar = new Car();  
    myCar.start();  
}  
}
```

9. Static Members

```
```java  
// This program demonstrates static members in Java
class StaticExample {
 static int counter = 0; // Static variable shared among all objects

 StaticExample() {
 counter++; // Increment counter when an object is created
 }

 static void displayCount() {
 System.out.println("Number of objects created: " + counter);
 }
}
```

```
public class StaticMemberExample {
 public static void main(String[] args) {
 new StaticExample(); // Creating an object
 new StaticExample(); // Another object
 new StaticExample(); // Another object

 StaticExample.displayCount(); // Calling static method
 }
}
...

```

### \*\*10. Final Keyword\*\*

```
```java  
// This program demonstrates the final keyword in Java  
class FinalExample {  
    final int MAX_VALUE = 100; // Final variable (cannot be changed)
```

```

    void display() {
        System.out.println("Max value is: " + MAX_VALUE);
    }
}

```

```

public class FinalKeywordExample {
    public static void main(String[] args) {
        FinalExample obj = new FinalExample();
        obj.display();
    }
}
...

```

11. Factorial of a Number Using Recursion

```

```java
// This program calculates factorial using recursion
class FactorialRecursion {
 static int factorial(int n) {
 if (n == 0 || n == 1) return 1; // Base case
 return n * factorial(n - 1); // Recursive call
 }

 public static void main(String[] args) {
 int num = 5;
 System.out.println("Factorial of " + num + " is: " + factorial(num));
 }
}
...

```

---

### \*\*12. Find the Factorial of a Given Number (Iterative)\*\*

```

```java
// This program calculates factorial using a loop
class FactorialIterative {
    static int factorial(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }
}

```

```

    }

    public static void main(String[] args) {
        int num = 6;
        System.out.println("Factorial of " + num + " is: " + factorial(num));
    }
}
...

```

13. Print the Output as 0, 1, 1, 2, 3 (Fibonacci Series)

```

```java
// This program prints the Fibonacci series up to a certain number
class FibonacciSeries {
 public static void main(String[] args) {
 int n = 5;
 int first = 0, second = 1;

 System.out.print("Fibonacci Series: " + first + ", " + second);
 for (int i = 2; i < n; i++) {
 int next = first + second;
 System.out.print(", " + next);
 first = second;
 second = next;
 }
 }
}
...

```

---

### \*\*14. Find the Biggest Number in a Given List\*\*

```

```java
// This program finds the biggest number in an array
import java.util.Arrays;

class BiggestNumber {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 65, 76, 5, 4, 33, 4, 34, 232, 3, 2323};
        int maxVal = Arrays.stream(numbers).max().getAsInt(); // Finds max value
        System.out.println("The biggest number is: " + maxVal);
    }
}

```

...

15. Read and Write a File

```java

// This program reads and writes to a file

import java.io.\*;

class FileReadWrite {

public static void main(String[] args) {

try {

    // Writing to a file

    FileWriter writer = new FileWriter("output.txt");

    writer.write("Hello, this is a sample text.");

    writer.close();

    // Reading from a file

    FileReader reader = new FileReader("output.txt");

    BufferedReader bufferedReader = new BufferedReader(reader);

    String line;

    while ((line = bufferedReader.readLine()) != null) {

        System.out.println(line);

    }

    bufferedReader.close();

} catch (IOException e) {

    System.out.println("An error occurred.");

    e.printStackTrace();

}

}

}

...

---

### \*\*16. Print a Triangle of Numbers\*\*

```java

// This program prints a triangle pattern of numbers

class NumberTriangle {

public static void main(String[] args) {

int rows = 5;

for (int i = 1; i <= rows; i++) {

 for (int j = 1; j <= i; j++) {

 System.out.print(j + " ");


```

    }
    System.out.println();
}
}
}
...

```

8. Demonstrate if...else Statement

// This program demonstrates the use of an if...else statement
import java.util.Scanner;

```

public class IfElseExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = scanner.nextInt();

        if (num > 0) {
            System.out.println("The number is positive.");
        } else {
            System.out.println("The number is non-positive.");
        }

        scanner.close();
    }
}

```

9. Demonstrate Nested if...else if... Statement

// This program demonstrates a nested if...else if statement
import java.util.Scanner;

```

public class NestedIfElseExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your marks: ");
    }
}

```

```
int marks = scanner.nextInt();

if (marks >= 90) {
    System.out.println("Grade: A");
} else if (marks >= 75) {
    System.out.println("Grade: B");
} else if (marks >= 50) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: F");
}

scanner.close();
}
}
```

10. Demonstrate Nested if...else Statement

// This program demonstrates nested if...else statements
import java.util.Scanner;

```
public class NestedIfElseExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = scanner.nextInt();

        if (num >= 0) {
            if (num == 0) {
                System.out.println("The number is zero.");
            } else {
                System.out.println("The number is positive.");
            }
        } else {
            System.out.println("The number is negative.");
        }

        scanner.close();
    }
}
```

15. Demonstrate for Loop

// This program prints numbers from 1 to 5 using a for loop

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Number: " + i);  
        }  
    }  
}
```

16. Print Stars Using for Loop (Triangle Pattern)

// This program prints a triangle pattern with stars

```
public class TrianglePattern {  
    public static void main(String[] args) {  
        int rows = 5;  
        for (int i = 1; i <= rows; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

17. Demonstrate While Loop

// This program prints numbers using a while loop

```
public class WhileLoopExample {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println("Value: " + i);  
            i++;  
        }  
    }  
}
```

18. Print Entered Number in Reverse

```
// This program reverses a given number
import java.util.Scanner;

public class ReverseNumber {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = scanner.nextInt();

        int reverse = 0;
        while (num > 0) {
            reverse = reverse * 10 + num % 10;
            num /= 10;
        }

        System.out.println("Reversed Number: " + reverse);
        scanner.close();
    }
}
```

19. Demonstrate the Usage of Break Statement Inside a While Loop

```
// This program exits the loop when it finds number 3
public class BreakExample {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            if (i == 3) {
                System.out.println("Breaking at number " + i);
                break;
            }
            System.out.println("Number: " + i);
            i++;
        }
    }
}
```

20. Print the Alphabets Using for Loop

```
// This program prints alphabets from A to Z
```

```
public class PrintAlphabets {
    public static void main(String[] args) {
        for (char ch = 'A'; ch <= 'Z'; ch++) {
            System.out.print(ch + " ");
        }
    }
}
```

21. Demonstrate the Usage of Break and Continue Statements Inside a While Loop

// This program demonstrates break and continue inside a while loop

```
public class BreakContinueExample {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            if (i == 3) {
                System.out.println("Skipping number 3");
                i++;
                continue; // Skip this iteration
            }
            System.out.println("Number: " + i);
            i++;
        }
    }
}
```

22. Demonstrate for-each Loop

// This program demonstrates the use of a for-each loop

```
public class ForEachExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        System.out.println("Numbers in array:");
        for (int num : numbers) {
            System.out.println(num); // Iterate through array elements
        }
    }
}
```

23. Print Floyd's Triangle

```
// This program prints Floyd's triangle
public class FloydsTriangle {
    public static void main(String[] args) {
        int rows = 5, num = 1;

        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(num + " ");
                num++; // Increment number
            }
            System.out.println();
        }
    }
}
```

24. Print Pascal's Triangle

```
// This program prints Pascal's triangle
public class PascalsTriangle {
    static int factorial(int num) {
        int fact = 1;
        for (int i = 1; i <= num; i++) {
            fact *= i;
        }
        return fact;
    }

    static int pascalValue(int row, int col) {
        return factorial(row) / (factorial(col) * factorial(row - col));
    }

    public static void main(String[] args) {
        int rows = 5;

        for (int i = 0; i < rows; i++) {
            for (int space = 0; space < rows - i; space++) {
                System.out.print(" "); // Print spaces
            }
        }
    }
}
```

```

        for (int j = 0; j <= i; j++) {
            System.out.print(pascalValue(i, j) + " "); // Print Pascal's values
        }
        System.out.println();
    }
}

```

25. Print the Fibonacci Series

// This program prints Fibonacci series up to a given number

```

public class FibonacciSeries {
    public static void main(String[] args) {
        int n = 10;
        int first = 0, second = 1;

        System.out.print("Fibonacci Series: " + first + ", " + second);
        for (int i = 2; i < n; i++) {
            int next = first + second;
            System.out.print(", " + next);
            first = second;
            second = next;
        }
    }
}

```

26. Printing the Multiplication Table

// This program prints the multiplication table of a given number

```

import java.util.Scanner;

public class MultiplicationTable {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number for multiplication table: ");
        int num = scanner.nextInt();

        System.out.println("Multiplication Table of " + num + " :");
        for (int i = 1; i <= 10; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
        }
    }
}

```

```
        scanner.close();
    }
}
```

27. Calculate the Power of a Number Using a While Loop

// This program calculates the power of a number using a while loop
import java.util.Scanner;

```
public class PowerCalculation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter base number: ");
        int base = scanner.nextInt();
        System.out.print("Enter exponent: ");
        int exponent = scanner.nextInt();

        int result = 1;
        int count = 0;
        while (count < exponent) {
            result *= base;
            count++;
        }

        System.out.println(base + "^" + exponent + " = " + result);
        scanner.close();
    }
}
```

28. Demonstrate Method Overloading

// This program demonstrates method overloading in Java
class OverloadingExample {
 void display(int num) {
 System.out.println("Integer: " + num);
 }

 void display(String text) {
 System.out.println("String: " + text);
 }
}


```

    void display(double num) {
        System.out.println("Double: " + num);
    }
}

public class MethodOverloadingExample {
    public static void main(String[] args) {
        OverloadingExample obj = new OverloadingExample();

        obj.display(10);    // Calls the method with integer parameter
        obj.display("Hello"); // Calls the method with string parameter
        obj.display(10.5);  // Calls the method with double parameter
    }
}

```

29. Demonstrate Method Overriding

// This program demonstrates method overriding in Java

```

class Parent {
    void showMessage() {
        System.out.println("Message from Parent class.");
    }
}

class Child extends Parent {
    @Override
    void showMessage() {
        System.out.println("Message from Child class.");
    }
}

public class MethodOverridingExample {
    public static void main(String[] args) {
        Parent parentObj = new Parent();
        Child childObj = new Child();

        parentObj.showMessage(); // Calls Parent class method
        childObj.showMessage();  // Calls overridden Child class method
    }
}

```

30. Demonstrate Inheritance

```
// This program demonstrates inheritance in Java
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.makeSound(); // Inherited method from Animal class
        myDog.bark();      // Method specific to Dog class
    }
}
```

31. Demonstrate the Use of the 'super' Keyword

```
// This program demonstrates the use of 'super' keyword in Java
class Parent {
    void displayMessage() {
        System.out.println("Message from Parent class.");
    }
}

class Child extends Parent {
    void displayMessage() {
        super.displayMessage(); // Calls the parent class method
        System.out.println("Message from Child class.");
    }
}

public class SuperKeywordExample {
    public static void main(String[] args) {
        Child childObj = new Child();
        childObj.displayMessage(); // Calls both parent and child method
    }
}
```

```
}  
}
```

32. Find the Largest Number in an Array

// This program finds the largest number in an array

```
import java.util.Arrays;
```

```
public class LargestNumberInArray {  
    public static void main(String[] args) {  
        int[] numbers = {12, 34, 55, 89, 21, 98, 43};  
        int maxVal = Arrays.stream(numbers).max().getAsInt(); // Find the max  
        System.out.println("The largest number is: " + maxVal);  
    }  
}
```

33. Remove Duplicates from an Array

// This program removes duplicates from an array

```
import java.util.Arrays;
```

```
import java.util.LinkedHashSet;
```

```
public class RemoveDuplicates {  
    public static void main(String[] args) {  
        Integer[] numbers = {1, 2, 2, 3, 4, 4, 5, 6, 6, 7};  
        LinkedHashSet<Integer> uniqueNumbers = new  
        LinkedHashSet<>(Arrays.asList(numbers));  
  
        System.out.println("Array without duplicates: " + uniqueNumbers);  
    }  
}
```

34. Find the Second Largest Number in an Array

// This program finds the second largest number in an array

```
import java.util.Arrays;
```

```
public class SecondLargestInArray {  
    public static void main(String[] args) {  
        int[] numbers = {12, 45, 89, 65, 43, 76};
```

```
        Arrays.sort(numbers);
        System.out.println("The second largest number is: " + numbers[numbers.length - 2]);
    }
}
```

35. Sort an Array Using Bubble Sort

```
// This program sorts an array using bubble sort
import java.util.Arrays;

public class BubbleSortExample {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) { // Swap if the element is greater
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int[] numbers = {5, 2, 9, 1, 6, 3};
        bubbleSort(numbers);
        System.out.println("Sorted array using Bubble Sort: " + Arrays.toString(numbers));
    }
}
```

36. Sort an Array Using Selection Sort

```
// This program sorts an array using selection sort
import java.util.Arrays;

public class SelectionSortExample {
    public static void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            int minIndex = i; // Assume the first element is the smallest
```

```

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j; // Find the index of the smallest element
            }
        }
        int temp = arr[minIndex]; // Swap smallest element with current element
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

public static void main(String[] args) {
    int[] numbers = {29, 10, 14, 37, 13};
    selectionSort(numbers);
    System.out.println("Sorted array using Selection Sort: " + Arrays.toString(numbers));
}
}

```

37. Implement Binary Search

// This program implements binary search to find an element
import java.util.Arrays;

```

public class BinarySearchExample {
    static int binarySearch(int[] arr, int key) {
        int left = 0, right = arr.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == key) return mid;
            if (arr[mid] < key) left = mid + 1;
            else right = mid - 1;
        }

        return -1; // Return -1 if the element is not found
    }

    public static void main(String[] args) {
        int[] numbers = {5, 12, 18, 22, 30, 42, 50};
        Arrays.sort(numbers); // Ensure array is sorted for binary search
        int key = 22;
    }
}

```

```

        int index = binarySearch(numbers, key);
        if (index != -1) {
            System.out.println(key + " found at index " + index);
        } else {
            System.out.println(key + " not found.");
        }
    }
}

```

38. Implement Linear Search

// This program implements linear search to find an element

```

public class LinearSearchExample {
    static int linearSearch(int[] arr, int key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) return i; // Return index if found
        }
        return -1; // Return -1 if not found
    }

    public static void main(String[] args) {
        int[] numbers = {3, 8, 12, 25, 35, 40};
        int key = 25;

        int index = linearSearch(numbers, key);
        if (index != -1) {
            System.out.println(key + " found at index " + index);
        } else {
            System.out.println(key + " not found.");
        }
    }
}

```

39. Java Program to Compare Two Strings

// This program compares two strings using equals() method

import java.util.Scanner;

```

public class CompareStrings {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}

```

```
System.out.print("Enter first string: ");
String str1 = scanner.nextLine();

System.out.print("Enter second string: ");
String str2 = scanner.nextLine();

if (str1.equals(str2)) {
    System.out.println("Strings are equal.");
} else {
    System.out.println("Strings are not equal.");
}

scanner.close();
}
}
```

40. Java Program to Reverse a String

```
// This program reverses a given string
import java.util.Scanner;

public class ReverseString {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = scanner.nextLine();

        String reversed = new StringBuilder(str).reverse().toString(); // Reverse the string
        System.out.println("Reversed String: " + reversed);

        scanner.close();
    }
}
```

41. Java Program to Count the Number of Vowels and Consonants in a String

```
// This program counts vowels and consonants in a string
import java.util.Scanner;
```

```

public class CountVowelsConsonants {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = scanner.nextLine().toLowerCase();

        int vowels = 0, consonants = 0;
        for (char ch : str.toCharArray()) {
            if (Character.isLetter(ch)) {
                if ("aeiou".indexOf(ch) != -1) {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }

        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);

        scanner.close();
    }
}

```

42. Java Program to Find the Frequency of Characters in a String

```

// This program finds the frequency of characters in a string
import java.util.HashMap;
import java.util.Scanner;

public class CharacterFrequency {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = scanner.nextLine();

        HashMap<Character, Integer> frequencyMap = new HashMap<>();

        for (char ch : str.toCharArray()) {
            frequencyMap.put(ch, frequencyMap.getOrDefault(ch, 0) + 1);
        }
    }
}

```



```
    }

    System.out.println("Character Frequencies: " + frequencyMap);

    scanner.close();
}
}
```

43. Demonstrate Exception Handling Using try-catch

// This program demonstrates exception handling using a try-catch block

```
public class TryCatchExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will cause an exception
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        }
    }
}
```

44. Demonstrate the Use of finally Block

// This program demonstrates the use of the finally block

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            System.out.println("Inside try block.");
            int num = 10 / 0; // Generates an exception
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

45. Demonstrate the Use of throw and throws

```
// This program demonstrates 'throw' and 'throws' keywords in Java
class ThrowExample {
    static void checkAge(int age) throws IllegalArgumentException {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be at least 18.");
        } else {
            System.out.println("Eligible to vote.");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(16); // This will throw an exception
        } catch (IllegalArgumentException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

46. Demonstrate Custom Exceptions

```
// This program demonstrates how to create and use custom exceptions
class CustomException extends Exception {
    CustomException(String message) {
        super(message);
    }
}

public class CustomExceptionExample {
    static void validate(int num) throws CustomException {
        if (num < 0) {
            throw new CustomException("Negative numbers are not allowed.");
        }
        System.out.println("Valid input: " + num);
    }

    public static void main(String[] args) {
        try {
            validate(-5);
        } catch (CustomException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
}
```

47. Demonstrate the Use of ArrayList

```
// This program demonstrates the use of ArrayList
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Printing the list
        System.out.println("Fruits: " + fruits);

        // Removing an element
        fruits.remove("Banana");

        // Checking if an element exists
        System.out.println("Contains Apple? " + fruits.contains("Apple"));
    }
}
```

48. Demonstrate the Use of HashMap

```
// This program demonstrates the use of HashMap
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {
        HashMap<String, Integer> scores = new HashMap<>();

        // Adding key-value pairs
        scores.put("Alice", 95);
        scores.put("Bob", 85);
        scores.put("Charlie", 90);
    }
}
```

```

// Getting value by key
System.out.println("Bob's score: " + scores.get("Bob"));

// Checking if a key exists
System.out.println("Contains Alice? " + scores.containsKey("Alice"));

// Iterating over keys and values
for (String key : scores.keySet()) {
    System.out.println(key + " -> " + scores.get(key));
}
}
}

```

49. Demonstrate the Use of HashSet

```

// This program demonstrates the use of HashSet
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<Integer> numbers = new HashSet<>();

        // Adding elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(10); // Duplicate, won't be added

        // Printing the set
        System.out.println("Numbers: " + numbers);

        // Checking if an element exists
        System.out.println("Contains 20? " + numbers.contains(20));

        // Removing an element
        numbers.remove(30);
        System.out.println("Numbers after removal: " + numbers);
    }
}

```

50. Demonstrate the Use of LinkedList

```
// This program demonstrates the use of LinkedList
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> cities = new LinkedList<>();

        // Adding elements
        cities.add("New York");
        cities.add("Paris");
        cities.add("Tokyo");

        // Printing the list
        System.out.println("Cities: " + cities);

        // Adding elements at specific positions
        cities.addFirst("London");
        cities.addLast("Sydney");

        System.out.println("Cities after updates: " + cities);
    }
}
```

51. Demonstrate the Use of TreeSet

```
// This program demonstrates the use of TreeSet
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<Integer> numbers = new TreeSet<>();

        // Adding elements
        numbers.add(50);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);

        // Printing the sorted set
        System.out.println("Sorted TreeSet: " + numbers);

        // Getting the smallest and largest elements
        System.out.println("Smallest element: " + numbers.first());
    }
}
```

```
        System.out.println("Largest element: " + numbers.last());
    }
}
```

52. Demonstrate the Use of PriorityQueue

// This program demonstrates the use of PriorityQueue

```
import java.util.PriorityQueue;
```

```
public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        // Adding elements
        pq.add(40);
        pq.add(20);
        pq.add(30);
        pq.add(10);

        // Printing and removing elements one by one
        while (!pq.isEmpty()) {
            System.out.println("Removed: " + pq.poll()); // Retrieves and removes the head element
        }
    }
}
```

53. Demonstrate the Use of Stack

// This program demonstrates the use of Stack

```
import java.util.Stack;
```

```
public class StackExample {
    public static void main(String[] args) {
        Stack<String> books = new Stack<>();

        // Adding elements
        books.push("Java Basics");
        books.push("Data Structures");
        books.push("Algorithms");

        // Popping elements
```

```
        System.out.println("Top book: " + books.peek());
        System.out.println("Removed book: " + books.pop());
        System.out.println("Remaining books: " + books);
    }
}
```

54. Demonstrate the Use of Vector

```
// This program demonstrates the use of Vector
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
        Vector<Integer> numbers = new Vector<>();

        // Adding elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        // Printing elements
        System.out.println("Numbers in vector: " + numbers);

        // Getting an element by index
        System.out.println("Element at index 1: " + numbers.get(1));
    }
}
```

55. Demonstrate the Use of Deque

```
// This program demonstrates the use of Deque (Double-ended queue)
import java.util.Deque;
import java.util.LinkedList;

public class DequeExample {
    public static void main(String[] args) {
        Deque<Integer> deque = new LinkedList<>();

        // Adding elements at both ends
        deque.addFirst(10);
        deque.addLast(20);
    }
}
```

```

deque.addFirst(5);
deque.addLast(30);

System.out.println("Deque: " + deque);

// Removing elements from both ends
System.out.println("Removed first element: " + deque.removeFirst());
System.out.println("Removed last element: " + deque.removeLast());

System.out.println("Deque after removals: " + deque);
}
}

```

56. Demonstrate the Use of LinkedHashMap

// This program demonstrates the use of LinkedHashMap (Maintains insertion order)

```

import java.util.LinkedHashMap;
import java.util.Map;

```

```

public class LinkedHashMapExample {
    public static void main(String[] args) {
        LinkedHashMap<String, Integer> items = new LinkedHashMap<>();

        // Adding key-value pairs
        items.put("Apple", 50);
        items.put("Banana", 30);
        items.put("Orange", 40);

        System.out.println("LinkedHashMap items:");
        for (Map.Entry<String, Integer> entry : items.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
    }
}

```

57. Demonstrate the Use of LinkedHashSet

// This program demonstrates the use of LinkedHashSet (Maintains insertion order)

```

import java.util.LinkedHashSet;

```

```

public class LinkedHashSetExample {

```



```

public static void main(String[] args) {
    LinkedHashSet<String> names = new LinkedHashSet<>();

    // Adding elements
    names.add("Alice");
    names.add("Bob");
    names.add("Charlie");
    names.add("Alice"); // Duplicate, won't be added

    System.out.println("LinkedHashSet: " + names);
}
}

```

58. Find Duplicates in a List

// This program finds duplicate elements in a list

```

import java.util.ArrayList;
import java.util.HashSet;

```

```

public class FindDuplicates {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(10); // Duplicate
        numbers.add(40);
        numbers.add(30); // Duplicate

        HashSet<Integer> uniqueNumbers = new HashSet<>();
        HashSet<Integer> duplicates = new HashSet<>();

        for (int num : numbers) {
            if (!uniqueNumbers.add(num)) { // If already exists, it's a duplicate
                duplicates.add(num);
            }
        }

        System.out.println("Duplicate numbers: " + duplicates);
    }
}

```

59. Compare Two HashMaps for Equality

// This program compares two HashMaps for equality

```
import java.util.HashMap;
```

```
public class CompareHashMaps {  
    public static void main(String[] args) {  
        HashMap<String, Integer> map1 = new HashMap<>();  
        HashMap<String, Integer> map2 = new HashMap<>();  
  
        // Adding key-value pairs  
        map1.put("A", 10);  
        map1.put("B", 20);  
        map1.put("C", 30);  
  
        map2.put("A", 10);  
        map2.put("B", 20);  
        map2.put("C", 30);  
  
        // Checking equality  
        System.out.println("Are both HashMaps equal? " + map1.equals(map2));  
    }  
}
```

60. Creating an ArrayList

// This program demonstrates creating an ArrayList

```
import java.util.ArrayList;
```

```
public class CreateArrayList {  
    public static void main(String[] args) {  
        ArrayList<String> names = new ArrayList<>();  
  
        // Adding elements  
        names.add("Alice");  
        names.add("Bob");  
        names.add("Charlie");  
  
        System.out.println("Names in ArrayList: " + names);  
    }  
}
```

61. Creating a LinkedList

```
// This program demonstrates creating a LinkedList
import java.util.LinkedList;

public class CreateLinkedList {
    public static void main(String[] args) {
        LinkedList<Integer> numbers = new LinkedList<>();

        // Adding elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        System.out.println("Numbers in LinkedList: " + numbers);
    }
}
```

62. Creating a HashSet

```
// This program demonstrates creating a HashSet
import java.util.HashSet;

public class CreateHashSet {
    public static void main(String[] args) {
        HashSet<String> colors = new HashSet<>();

        // Adding elements
        colors.add("Red");
        colors.add("Green");
        colors.add("Blue");
        colors.add("Red"); // Duplicate, won't be added

        System.out.println("Colors in HashSet: " + colors);
    }
}
```

63. Creating a LinkedHashSet

```
// This program demonstrates creating a LinkedHashSet (Maintains insertion order)
import java.util.LinkedHashSet;
```

```
public class LinkedHashSetExample {
    public static void main(String[] args) {
        LinkedHashSet<Integer> numbers = new LinkedHashSet<>();

        // Adding elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(10); // Duplicate, won't be added

        System.out.println("LinkedHashSet: " + numbers);
    }
}
```

64. Creating a TreeSet

// This program demonstrates creating a TreeSet (Stores elements in sorted order)
import java.util.TreeSet;

```
public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<String> names = new TreeSet<>();

        // Adding elements
        names.add("John");
        names.add("Alice");
        names.add("Bob");

        // TreeSet maintains sorted order
        System.out.println("Sorted TreeSet: " + names);

        // Checking first and last elements
        System.out.println("First element: " + names.first());
        System.out.println("Last element: " + names.last());
    }
}
```

65. Creating a PriorityQueue

// This program demonstrates creating a PriorityQueue (Sorted in natural order)
import java.util.PriorityQueue;

```

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        // Adding elements
        pq.add(30);
        pq.add(10);
        pq.add(20);
        pq.add(40);

        System.out.println("PriorityQueue contents:");
        while (!pq.isEmpty()) {
            System.out.println(pq.poll()); // Retrieves and removes the head element
        }
    }
}

```

66. Creating a HashMap Using the Map Interface

// This program demonstrates creating a HashMap using the Map interface

```

import java.util.HashMap;
import java.util.Map;

public class MapExample {
    public static void main(String[] args) {
        Map<String, Integer> products = new HashMap<>();

        // Adding key-value pairs
        products.put("Laptop", 1000);
        products.put("Phone", 700);
        products.put("Tablet", 500);

        // Iterating over the Map
        for (Map.Entry<String, Integer> entry : products.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
    }
}

```

67. Creating a TreeMap

```
// This program demonstrates creating a TreeMap (Automatically sorts keys)
import java.util.TreeMap;
```

```
public class TreeMapExample {
    public static void main(String[] args) {
        TreeMap<String, Integer> students = new TreeMap<>();

        // Adding key-value pairs
        students.put("Alice", 95);
        students.put("Bob", 85);
        students.put("Charlie", 90);

        // TreeMap maintains keys in sorted order
        System.out.println("TreeMap contents:");
        for (String key : students.keySet()) {
            System.out.println(key + " -> " + students.get(key));
        }
    }
}
```

68. Creating a LinkedHashMap

```
// This program demonstrates creating a LinkedHashMap (Maintains insertion order)
import java.util.LinkedHashMap;
import java.util.Map;
```

```
public class LinkedHashMapExample {
    public static void main(String[] args) {
        LinkedHashMap<String, Integer> productPrices = new LinkedHashMap<>();

        // Adding elements
        productPrices.put("Laptop", 1000);
        productPrices.put("Phone", 700);
        productPrices.put("Tablet", 500);

        // Maintaining order of insertion
        System.out.println("LinkedHashMap contents:");
        for (Map.Entry<String, Integer> entry : productPrices.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
    }
}
```

69. Creating a TreeMap for Sorting Numbers

```
// This program sorts numbers using TreeMap
import java.util.TreeMap;

public class SortedTreeMapExample {
    public static void main(String[] args) {
        TreeMap<Integer, String> rankList = new TreeMap<>();

        // Adding elements
        rankList.put(3, "Charlie");
        rankList.put(1, "Alice");
        rankList.put(2, "Bob");

        System.out.println("Sorted TreeMap contents:");
        for (Integer key : rankList.keySet()) {
            System.out.println("Rank " + key + " -> " + rankList.get(key));
        }
    }
}
```

70. Creating a TreeMap Using Custom Sorting (Comparator)

```
// This program demonstrates a custom sorting order in TreeMap
import java.util.Comparator;
import java.util.TreeMap;

public class CustomTreeMapSorting {
    public static void main(String[] args) {
        TreeMap<String, Integer> scores = new TreeMap<>(Comparator.reverseOrder()); // Sort in
        descending order

        // Adding elements
        scores.put("Alice", 95);
        scores.put("Bob", 85);
        scores.put("Charlie", 90);

        System.out.println("TreeMap with reverse sorting:");
        for (String key : scores.keySet()) {
            System.out.println(key + " -> " + scores.get(key));
        }
    }
}
```

```
}  
}
```

71. Creating a Basic Thread Using Thread Class

// This program demonstrates creating a basic thread using the Thread class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running...");  
    }  
}
```

```
public class ThreadExample {  
    public static void main(String[] args) {  
        MyThread thread1 = new MyThread();  
        thread1.start(); // Start the thread  
    }  
}
```

72. Creating a Thread Using Runnable Interface

// This program demonstrates creating a thread using the Runnable interface

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Thread is executing...");  
    }  
}
```

```
public class RunnableExample {  
    public static void main(String[] args) {  
        Thread thread2 = new Thread(new MyRunnable());  
        thread2.start(); // Start the thread  
    }  
}
```

73. Demonstrating Thread Synchronization

// This program demonstrates thread synchronization to avoid race conditions

```
class SharedResource {  
    synchronized void printNumbers(int n) {
```



```

        for (int i = 1; i <= 5; i++) {
            System.out.println(n * i);
            try { Thread.sleep(500); } catch (InterruptedException e) { System.out.println(e); }
        }
    }
}

class MyThread extends Thread {
    SharedResource resource;

    MyThread(SharedResource res) {
        this.resource = res;
    }

    public void run() {
        resource.printNumbers(5);
    }
}

public class ThreadSyncExample {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        MyThread t1 = new MyThread(resource);
        MyThread t2 = new MyThread(resource);

        t1.start();
        t2.start();
    }
}

```

74. Demonstrating Thread Priority

// This program demonstrates setting priorities for threads

```

class PriorityThread extends Thread {
    public void run() {
        System.out.println(getName() + " running with priority " + getPriority());
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        PriorityThread t1 = new PriorityThread();
    }
}

```

```
PriorityThread t2 = new PriorityThread();
PriorityThread t3 = new PriorityThread();

t1.setPriority(Thread.MIN_PRIORITY);
t2.setPriority(Thread.MAX_PRIORITY);
t3.setPriority(Thread.NORM_PRIORITY);

t1.start();
t2.start();
t3.start();
}
}
```

75. Writing to a File

```
// This program writes data to a file
import java.io.FileWriter;
import java.io.IOException;

public class FileWriteExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("output.txt");
            writer.write("Hello, this is a sample text file.");
            writer.close();
            System.out.println("File written successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

76. Reading from a File

```
// This program reads data from a file
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class FileReadExample {
    public static void main(String[] args) {
```

```

try {
    FileReader reader = new FileReader("output.txt");
    BufferedReader bufferedReader = new BufferedReader(reader);
    String line;

    while ((line = bufferedReader.readLine()) != null) {
        System.out.println(line);
    }

    bufferedReader.close();
} catch (IOException e) {
    System.out.println("An error occurred: " + e.getMessage());
}
}

```

77. Append Data to a File

// This program appends data to an existing file

```
import java.io.FileWriter;
```

```
import java.io.IOException;
```

```

public class FileAppendExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("output.txt", true); // 'true' for append mode
            writer.write("\nThis is an appended line.");
            writer.close();
            System.out.println("Data appended successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}

```

78. Checking If a File Exists

// This program checks whether a file exists

```
import java.io.File;
```

```
public class FileExistsExample {
```

```

public static void main(String[] args) {
    File file = new File("output.txt");

    if (file.exists()) {
        System.out.println("File exists.");
    } else {
        System.out.println("File does not exist.");
    }
}
}

```

79. Establish a Connection to a MySQL Database

// This program demonstrates connecting to a MySQL database using JDBC

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

```

```

public class DatabaseConnection {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase"; // Change database name
        String username = "root"; // Your database username
        String password = "password"; // Your database password

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            System.out.println("Connected to the database successfully!");
            conn.close();
        } catch (SQLException e) {
            System.out.println("Error connecting to database: " + e.getMessage());
        }
    }
}

```

80. Creating a Table in MySQL Using JDBC

// This program demonstrates creating a table using JDBC

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

```

```

public class CreateTable {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "root";
        String password = "password";

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            Statement stmt = conn.createStatement();

            String sql = "CREATE TABLE Students (id INT PRIMARY KEY, name VARCHAR(50),
age INT)";
            stmt.executeUpdate(sql);

            System.out.println("Table created successfully!");
            conn.close();
        } catch (SQLException e) {
            System.out.println("Error creating table: " + e.getMessage());
        }
    }
}

```

81. Insert Data into a Database Table

// This program demonstrates inserting data into a database table

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class InsertData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "root";
        String password = "password";

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            String sql = "INSERT INTO Students (id, name, age) VALUES (?, ?, ?)";
            PreparedStatement pstmt = conn.prepareStatement(sql);

            pstmt.setInt(1, 1);

```

```

        pstmt.setString(2, "John Doe");
        pstmt.setInt(3, 22);

        pstmt.executeUpdate();
        System.out.println("Data inserted successfully!");

        conn.close();
    } catch (SQLException e) {
        System.out.println("Error inserting data: " + e.getMessage());
    }
}
}

```

82. Retrieve Data from a Database

```

// This program demonstrates retrieving data from a database
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class RetrieveData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "root";
        String password = "password";

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM Students");

            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") + ", Name: " + rs.getString("name") + ", Age: "
+ rs.getInt("age"));
            }

            conn.close();
        } catch (SQLException e) {
            System.out.println("Error retrieving data: " + e.getMessage());
        }
    }
}

```

```
}
```

83. Updating Data in a Database Table

```
// This program updates a record in the database using JDBC
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class UpdateData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "root";
        String password = "password";

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            String sql = "UPDATE Students SET age = ? WHERE id = ?";
            PreparedStatement pstmt = conn.prepareStatement(sql);

            pstmt.setInt(1, 23); // New age
            pstmt.setInt(2, 1); // ID of student to update

            pstmt.executeUpdate();
            System.out.println("Data updated successfully!");

            conn.close();
        } catch (SQLException e) {
            System.out.println("Error updating data: " + e.getMessage());
        }
    }
}
```

84. Deleting Data from a Database Table

```
// This program deletes a record from the database using JDBC
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
```

```

public class DeleteData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "root";
        String password = "password";

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            String sql = "DELETE FROM Students WHERE id = ?";
            PreparedStatement pstmt = conn.prepareStatement(sql);

            pstmt.setInt(1, 1); // ID of student to delete

            pstmt.executeUpdate();
            System.out.println("Data deleted successfully!");

            conn.close();
        } catch (SQLException e) {
            System.out.println("Error deleting data: " + e.getMessage());
        }
    }
}

```

85. Using Transactions in JDBC

// This program demonstrates transaction handling in JDBC

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class TransactionExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "root";
        String password = "password";

        try {
            Connection conn = DriverManager.getConnection(url, username, password);
            conn.setAutoCommit(false); // Disable auto-commit

            Statement stmt = conn.createStatement();

```



```

stmt.executeUpdate("INSERT INTO Students VALUES (2, 'Jane Doe', 20)");
stmt.executeUpdate("INSERT INTO Students VALUES (3, 'Sam Smith', 22)");

conn.commit(); // Commit transaction
System.out.println("Transaction committed successfully!");

conn.close();
} catch (SQLException e) {
    System.out.println("Error executing transaction: " + e.getMessage());
}
}
}

```

Control Flow Statements

93. Verify Whether a Number Is a Perfect Number

// This program checks if a number is a perfect number
import java.util.Scanner;

```

public class PerfectNumberChecker {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = scanner.nextInt();

        int sum = 0;
        for (int i = 1; i <= num / 2; i++) {
            if (num % i == 0) {
                sum += i;
            }
        }

        if (sum == num) {
            System.out.println(num + " is a perfect number.");
        } else {
            System.out.println(num + " is not a perfect number.");
        }

        scanner.close();
    }
}

```

94. Print the Fibonacci Series from 1 to 10

```
// This program prints the Fibonacci series from 1 to 10
public class FibonacciSeries {
    public static void main(String[] args) {
        int first = 0, second = 1;

        System.out.print("Fibonacci Series: " + first + ", " + second);
        for (int i = 2; i < 10; i++) {
            int next = first + second;
            System.out.print(", " + next);
            first = second;
            second = next;
        }
    }
}
```

95. Find the Greatest of Three Numbers

```
// This program finds the greatest of three numbers
import java.util.Scanner;

public class GreatestOfThree {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int num1 = scanner.nextInt();
        System.out.print("Enter second number: ");
        int num2 = scanner.nextInt();
        System.out.print("Enter third number: ");
        int num3 = scanner.nextInt();

        int greatest = (num1 > num2) ? (num1 > num3 ? num1 : num3) : (num2 > num3 ? num2 :
num3);

        System.out.println("The greatest number is: " + greatest);
        scanner.close();
    }
}
```

96. Print the Largest Two Numbers from an Array

// This program finds the largest two numbers in an array

```
import java.util.Arrays;
```

```
public class LargestTwoNumbers {  
    public static void main(String[] args) {  
        int[] numbers = {12, 45, 89, 65, 43, 76};  
  
        Arrays.sort(numbers);  
        System.out.println("Largest numbers: " + numbers[numbers.length - 1] + " and " +  
numbers[numbers.length - 2]);  
    }  
}
```

Method Overloading and Overriding

97. Overload a Final Method

// Overloading a final method (allowed)

```
class FinalMethodExample {  
    final void show(int num) {  
        System.out.println("Integer: " + num);  
    }  
  
    void show(String text) { // Overloaded method with a different parameter  
        System.out.println("String: " + text);  
    }  
}  
  
public class FinalMethodOverloading {  
    public static void main(String[] args) {  
        FinalMethodExample obj = new FinalMethodExample();  
        obj.show(10);  
        obj.show("Hello");  
    }  
}
```

98. Overload a Static Method

```
// Overloading static method
class StaticMethodOverload {
    static void display(int num) {
        System.out.println("Integer: " + num);
    }

    static void display(double num) {
        System.out.println("Double: " + num);
    }
}

public class StaticMethodExample {
    public static void main(String[] args) {
        StaticMethodOverload.display(10);
        StaticMethodOverload.display(15.5);
    }
}
```

99. Override a Final Method (Not Allowed)

```
// This program demonstrates that overriding a final method is NOT allowed
class Parent {
    final void displayMessage() {
        System.out.println("Message from Parent class.");
    }
}

// This will cause a compilation error!
// class Child extends Parent {
//     void displayMessage() {
//         System.out.println("Message from Child class.");
//     }
// }
```

Explanation: A final method **cannot be overridden** in the subclass.

100. Override a Static Method (Method Hiding)

```
// This program demonstrates method hiding with static methods
```

```

class Parent {
    static void show() {
        System.out.println("Static method in Parent class.");
    }
}

class Child extends Parent {
    static void show() {
        System.out.println("Static method in Child class.");
    }
}

public class StaticMethodHiding {
    public static void main(String[] args) {
        Parent.show();
        Child.show();
    }
}

```

Exception Handling

101. Demonstrate the Advantage of Finally in Exception Handling

```

// This program demonstrates the advantage of finally block
public class FinallyExample {
    public static void main(String[] args) {
        try {
            System.out.println("Inside try block.");
            int num = 10 / 0; // Generates an exception
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed, ensuring resource cleanup.");
        }
    }
}

```

102. Create a Custom Immutable Class

```

// This program creates a custom immutable class
final class ImmutableExample {

```

```

private final String name;
private final int age;

public ImmutableExample(String name, int age) {
    this.name = name;
    this.age = age;
}

public String getName() {
    return name;
}

public int getAge() {
    return age;
}
}

public class ImmutableClassDemo {
    public static void main(String[] args) {
        ImmutableExample obj = new ImmutableExample("Alice", 25);
        System.out.println("Name: " + obj.getName());
        System.out.println("Age: " + obj.getAge());
    }
}

```

Arrays

103. Sort an Array Using Insertion Sort

// This program sorts an array using insertion sort
import java.util.Arrays;

```

public class InsertionSortExample {
    public static void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) { // Shift elements to right
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }
}

```

```

        }
        arr[j + 1] = key;
    }
}

public static void main(String[] args) {
    int[] numbers = {29, 10, 14, 37, 13};
    insertionSort(numbers);
    System.out.println("Sorted array using Insertion Sort: " + Arrays.toString(numbers));
}
}

```

104. Find the Sum of Elements in an Array

```

// This program finds the sum of elements in an array
public class ArraySum {
    public static void main(String[] args) {
        int[] numbers = {5, 10, 15, 20, 25};
        int sum = 0;

        for (int num : numbers) {
            sum += num;
        }

        System.out.println("Sum of array elements: " + sum);
    }
}

```

105. Find the Average of Elements in an Array

```

// This program calculates the average of elements in an array
public class ArrayAverage {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        int sum = 0;

        for (int num : numbers) {
            sum += num;
        }

        double average = sum / (double) numbers.length;
    }
}

```

```
        System.out.println("Average of array elements: " + average);
    }
}
```

106. Find the Maximum and Minimum Elements in an Array

// This program finds the maximum and minimum elements in an array
import java.util.Arrays;

```
public class MaxMinInArray {
    public static void main(String[] args) {
        int[] numbers = {23, 45, 12, 67, 34, 89};

        int max = Arrays.stream(numbers).max().getAsInt();
        int min = Arrays.stream(numbers).min().getAsInt();

        System.out.println("Maximum number: " + max);
        System.out.println("Minimum number: " + min);
    }
}
```

107. Find Duplicate Elements in an Array

// This program finds duplicate elements in an array
import java.util.HashSet;

```
public class FindDuplicates {
    public static void main(String[] args) {
        int[] numbers = {5, 10, 15, 5, 20, 15, 25};

        HashSet<Integer> uniqueNumbers = new HashSet<>();
        HashSet<Integer> duplicates = new HashSet<>();

        for (int num : numbers) {
            if (!uniqueNumbers.add(num)) { // If not added, it is duplicate
                duplicates.add(num);
            }
        }

        System.out.println("Duplicate numbers: " + duplicates);
    }
}
```



```
}
```

108. Swap Two Strings Without Using a Temporary Variable

// This program swaps two strings without using a temporary variable

```
public class SwapStrings {  
    public static void main(String[] args) {  
        String str1 = "Hello";  
        String str2 = "World";  
  
        System.out.println("Before Swap: str1 = " + str1 + ", str2 = " + str2);  
  
        str1 = str1 + str2; // Concatenate both strings  
        str2 = str1.substring(0, str1.length() - str2.length());  
        str1 = str1.substring(str2.length());  
  
        System.out.println("After Swap: str1 = " + str1 + ", str2 = " + str2);  
    }  
}
```

109. Find the Greatest Number in an Integer Array

// This program finds the greatest number in an integer array

```
import java.util.Arrays;  
  
public class GreatestNumberInArray {  
    public static void main(String[] args) {  
        int[] numbers = {23, 45, 12, 67, 34, 89};  
        int max = Arrays.stream(numbers).max().getAsInt();  
  
        System.out.println("Greatest number: " + max);  
    }  
}
```

String Handling

110. Check If Two Strings Are Anagrams

// This program checks if two strings are anagrams

```

import java.util.Arrays;

public class AnagramChecker {
    static boolean areAnagrams(String str1, String str2) {
        char[] arr1 = str1.toLowerCase().toCharArray();
        char[] arr2 = str2.toLowerCase().toCharArray();

        Arrays.sort(arr1);
        Arrays.sort(arr2);

        return Arrays.equals(arr1, arr2);
    }

    public static void main(String[] args) {
        String str1 = "listen";
        String str2 = "silent";

        System.out.println(str1 + " and " + str2 + " are anagrams? " + areAnagrams(str1, str2));
    }
}

```

111. Reverse the Words in a Sentence

```

// This program reverses the words in a sentence
public class ReverseWords {
    public static void main(String[] args) {
        String sentence = "Java is fun";
        String[] words = sentence.split(" ");
        StringBuilder reversedSentence = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            reversedSentence.append(words[i]).append(" ");
        }

        System.out.println("Reversed sentence: " + reversedSentence.toString().trim());
    }
}

```

112. Extract Numbers from a String and Print Only Alphabets

```

// This program extracts numbers from a string and prints only alphabets

```

```
public class ExtractAlphabets {  
    public static void main(String[] args) {  
        String str = "Java123Programming456";  
  
        String letters = str.replaceAll("[0-9]", ""); // Remove numbers  
        System.out.println("Letters only: " + letters);  
    }  
}
```

113. Find the Frequency of Words in a Paragraph

// This program finds the frequency of words in a paragraph

```
import java.util.HashMap;
```

```
public class WordFrequency {  
    public static void main(String[] args) {  
        String paragraph = "Java is powerful. Java is flexible. Java is popular.";  
        String[] words = paragraph.toLowerCase().replaceAll("[^a-zA-Z ]", "").split("\\s+");  
  
        HashMap<String, Integer> wordCount = new HashMap<>();  
  
        for (String word : words) {  
            wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);  
        }  
  
        System.out.println("Word frequencies: " + wordCount);  
    }  
}
```
